UNIVERSIDAD DE ANTIOQUIA
FACULTAD DE INGENIERÍA

INFORMÁTICA II – SEMESTRE 2026-2

DESAFÍO II: AMIGOS PARA SIEMPRE, CON UDE@BOOK

INFORME FINAL

Análisis, diseño e implementación

Autores: Brayan Camilo Silva Porras
Juan José Arboleda Cardona

Asignatura: Informática II
Docente: Aníbal Guerra

Medellín, Antioquia, Colombia
Octubre de 2026

Índice

1. Contexto del problema y análisis **2**

1.1. Información almacenada 2

1.2. Funcionalidades 2

1.3. Restricciones y reglas de funcionamiento 2

2. Estructuras de datos **3**

2.1. Matriz de amistad almacenada mediante bits 3

3. Diagrama de clases **4**

3.1. Diagrama UML resumido 4

3.2. Responsabilidad de cada clase 4

3.3. Relaciones entre clases 4

4. Formato de los archivos **6**

5. Algoritmos principales **6**

5.1. Carga de datos e inicio de sesión 6

5.2. Selección aleatoria de fotos de perfil con prioridad 7

5.3. Solicitudes de amistad 8

5.4. Sugerencias de amistad 8

5.5. Carrusel de fotos 8

5.6. Mensajes 8

5.7. Política de guardado 9

6. Medición del consumo de recursos **9**

7. Resultados **10**

8. Problemas de desarrollo **11**

9. Evolución de la solución **11**

10. Conclusiones **12**

1. Contexto del problema y análisis

Ude@Book es una red social cuya capa lógica se desarrolla en C++, mediante programación orientada a objetos e interacción por consola. Los datos se cargan desde archivos de texto al iniciar el programa y los cambios se guardan en los archivos correspondientes durante la ejecución.

1.1. Información almacenada

- **Usuarios:** nombre de usuario único, contraseña, correo electrónico, ciudad, país, fecha de nacimiento, fecha de inscripción y ruta de la foto de perfil.
- **Amistades:** relaciones bidireccionales entre usuarios.
- **Mensajes:** cada usuario mantiene referencias a los mensajes enviados y recibidos, ordenados por fecha. Cada mensaje tiene un estado de lectura.
- **Solicitudes de amistad:** solicitudes recibidas por un usuario, que pueden aceptarse o rechazarse.
- **Álbumes y fotos:** cada álbum tiene un nivel de privacidad (público, solo para amigos o privado). De cada foto se almacena la fecha de captura, el tamaño, la ruta y el número de "me gusta".

1.2. Funcionalidades

N.º	Funcionalidad	Descripción
I	Carga y guardado de datos	Lee los archivos al iniciar el programa y guarda los cambios durante la ejecución. No aparece como opción en el menú del usuario.
II	Inicio de sesión y mejores amigos	Permite iniciar sesión con un máximo de tres intentos y muestra tres fotos de perfil de amigos seleccionados aleatoriamente, con una ponderación basada en los amigos en común. El grupo puede cambiarse una vez.
III	Administración de amistades	Permite enviar solicitudes de amistad y aceptar o rechazar las solicitudes recibidas.
IV	Visita al perfil de un amigo	Muestra los álbumes públicos y los destinados a amigos, e incluye un carrusel circular de fotos con la opción de dar "me gusta".
V	Sugerencias de amistad	Presenta K sugerencias ($1 \leq K \leq 20$), ordenadas según la cantidad de amigos en común, con la posibilidad de enviar solicitudes.
VI	Administración de mensajes	Permite enviar mensajes de hasta 512 caracteres, consultar los mensajes no leídos y buscar mensajes por rango de fechas.
VII	Medición de recursos	Al finalizar cada funcionalidad, muestra las iteraciones contabilizadas, la memoria estimada y el uso de componentes externos.

Tabla 1: Funcionalidades contempladas en el enunciado.

1.3. Restricciones y reglas de funcionamiento

- No se utilizan contenedores de la biblioteca estándar de plantillas (STL), estructuras `struct` ni herencia. Las estructuras de datos se implementan mediante clases propias y, cuando corresponde, memoria dinámica.

- Los archivos se utilizan para cargar y guardar información; el procesamiento de los datos se realiza en memoria.
- No se puede enviar una solicitud de amistad a uno mismo, a un usuario que ya sea amigo, a quien ya haya enviado una solicitud al usuario actual ni a quien ya haya recibido una solicitud de este.
- Solo se permite enviar mensajes a los amigos. Cada mensaje debe contener entre 1 y 512 caracteres.
- Solo se pueden visitar los perfiles de los amigos y no se deben mostrar sus álbumes privados.
- La eliminación de un "me gusta" no debe producir un conteo negativo.
- En las búsquedas por rango de fechas, la fecha inicial no puede ser posterior a la fecha final.

2. Estructuras de datos

El programa debe admitir al menos 1000 usuarios y la eficiencia forma parte de los criterios de evaluación. Por ello, se seleccionaron estructuras que permiten reducir el costo de las operaciones frecuentes y controlar el uso de memoria.

Estructura	**Uso**	**Justificación**
`ListaDinamica<T>` (plantilla)	Usuarios, álbumes, fotos, solicitudes, mensajes y amigos.	Arreglo dinámico que duplica su capacidad cuando se llena. Ofrece acceso por índice en $O(1)$ e inserción al final con costo amortizado $O(1)$.
Arreglo de usuarios ordenado por nombre	Inicio de sesión y búsqueda de usuarios.	Permite realizar búsquedas binarias en $O(\log N)$. Para 1000 usuarios, una búsqueda requiere como máximo 10 comparaciones en el caso ideal de una implementación binaria convencional con ese tamaño de entrada.
`MatrizAmistad`	Consultar amistades y contar amigos en común.	Matriz de adyacencia almacenada mediante bits. La consulta de una relación tiene costo $O(1)$, mientras que los amigos en común se calculan mediante una operación AND entre filas.
Lista de amigos en cada `Usuario`	Recorrer las amistades de una persona.	Permite recorrer únicamente los amigos registrados, sin examinar todos los usuarios. Es ventajosa cuando la cantidad de amigos es pequeña en comparación con N.
Objeto `Mensaje` compartido	Almacenar mensajes enviados y recibidos.	El emisor es responsable de liberar el objeto y el receptor mantiene una referencia al mismo mensaje. Así se evita duplicar el texto y se comparte un único estado de lectura.

Tabla 2: Estructuras de datos seleccionadas para la solución.

2.1. Matriz de amistad almacenada mediante bits

Cada palabra de 64 bits (`unsigned long long`) representa 64 posiciones de una fila de la matriz. Para N usuarios, cada fila requiere

$$P = \left\lfloor \frac{N + 63}{64} \right\rfloor \tag{1}$$

palabras de 64 bits.

Para $N = 1000$, se obtiene $P = 16$. Por tanto, el almacenamiento de los bits ocupa

$$1000 \cdot 16 \cdot 8 = 128\,000 \text{ bytes}$$

equivalentes aproximadamente a 125 KiB. Este valor corresponde al arreglo de bits, sin incluir los metadatos ni las demás estructuras del programa.

Para consultar si los usuarios i y j son amigos, se obtiene el bit j mód 64 de la palabra $\lfloor j/64 \rfloor$ de la fila i:

$$\text{sonAmigos}(i, j) = \left(\text{bits} \left[iP + \left\lfloor \frac{j}{64} \right\rfloor \right] \gg (j \bmod 64) \right) \;\&\; 1 \tag{2}$$

Para determinar los amigos en común de dos usuarios, se realiza una operación AND palabra por palabra entre sus filas y se cuentan los bits encendidos. Este conteo puede optimizarse eliminando, en cada iteración, el bit encendido de menor posición mediante la operación $x \leftarrow x \,\&\, (x - 1)$.

3. Diagrama de clases

El diseño contempla 12 clases, sin utilizar herencia. Ocho representan entidades o responsabilidades directamente relacionadas con el problema: `UdeaBook`, `Usuario`, `SolicitudAmistad`, `Mensaje`, `Album`, `Foto`, `Fecha` y `medidorrecursos`. Las cuatro clases de apoyo son `ListaDinamica<T>`, `MatrizAmistad`, `CargadorDatos` y `GuardadorDatos`.

3.1. Diagrama UML resumido

3.2. Responsabilidad de cada clase

3.3. Relaciones entre clases

- **Composición:** `UdeaBook` es responsable de los usuarios y de la matriz de amistad. Cada usuario es responsable de sus álbumes, mensajes enviados y solicitudes recibidas. Cada álbum es responsable de sus fotos.
- **Asociación reflexiva:** existe una relación entre `Usuario` y `Usuario` que representa la amistad. Es bidireccional y admite de cero a muchos amigos por usuario.
- **Referencias compartidas:** el receptor almacena un puntero al mismo mensaje que conserva el emisor en su lista de enviados. La solicitud identifica al emisor mediante un índice.
- **Dependencia:** `UdeaBook` utiliza `medidorrecursos`, `CargadorDatos` y `GuardadorDatos` para coordinar las mediciones y la persistencia de la información.

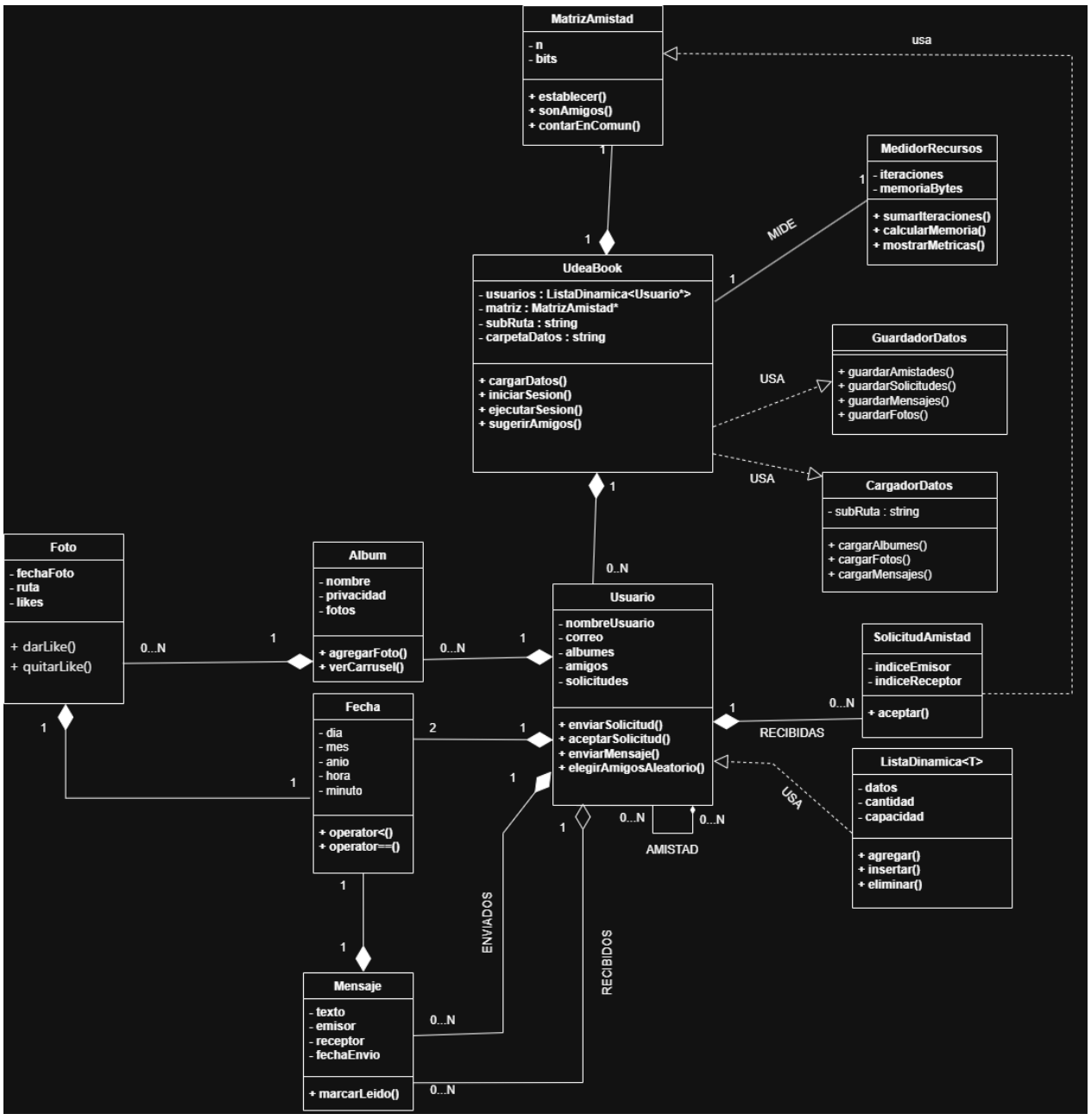


Figura 1: Diagrama UML resumido

Clase	Responsabilidad
`UdeaBook`	Clase principal. Administra la carga de usuarios, amistades y solicitudes; gestiona el inicio de sesión y las sugerencias; coordina los menús y determina cuándo guardar los cambios.
`Usuario`	Almacena los datos de una persona y sus listas. Participa en la validación y el envío de solicitudes y mensajes, y selecciona los amigos que aparecen en las fotos de perfil.
`SolicitudAmistad`	Almacena los índices del emisor y del receptor. Cuando la solicitud se acepta, se registra la relación de amistad en la matriz correspondiente.
`Mensaje`	Almacena el texto, el emisor, el receptor, la fecha y el estado de lectura. El texto está limitado a 512 caracteres y la clase sobrecarga `operator<` para comparar mensajes por fecha.
`Album` y `Foto`	El álbum almacena su nombre, ruta, privacidad y colección de fotos, además de gestionar el carrusel. La foto almacena su fecha, tamaño, ruta, número de "me gusta" y la información del último usuario que interactuó con ella.
`Fecha`	Representa el año, mes, día, hora y minuto. Sobrecarga los operadores `<` y `==`, así como el operador de salida `operator«`.
`medidorrecursos`	Registra las iteraciones y las llamadas a componentes externos, estima el consumo de memoria y presenta las métricas obtenidas.
`ListaDinamica<T>`	Plantilla que implementa el arreglo dinámico utilizado por las diferentes colecciones del programa.
`MatrizAmistad`	Implementa la matriz de adyacencia mediante bits y permite consultar relaciones y calcular amigos en común.
`CargadorDatos`	Lee los archivos `albums.txt`, `fotos.txt` y `mensajes.txt`.
`GuardadorDatos`	Escribe los archivos `amistades.txt`, `solicitudes.txt`, `mensajes.txt` y `fotos.txt`.

Tabla 3: Responsabilidades de las clases del sistema.

4. Formato de los archivos

Todos los archivos de datos se encuentran en la carpeta `datos`. Cada registro ocupa una línea y sus campos se separan mediante el carácter `|`. Las fechas completas se representan con el formato `AAAAMMDDHHMM`, de 12 dígitos. Por ejemplo, `202610101530` representa el 10 de octubre de 2026 a las 15:30.

Los registros con formato incorrecto en los archivos de álbumes, fotos y mensajes se omiten. En los archivos de usuarios, amistades y solicitudes, los errores de formato se notifican por consola. Los archivos solo se reescriben cuando existen cambios pendientes, de acuerdo con la política de guardado descrita posteriormente.

5. Algoritmos principales

5.1. Carga de datos e inicio de sesión

1. Se lee la subruta de las fotos y se crea la lista de usuarios a partir de la cantidad indicada en la primera línea del archivo. Se verifica que los nombres estén ordenados. Posteriormente, se inicializa la matriz de amistad con ceros.

2. Para cada amistad se buscan ambos usuarios mediante búsqueda binaria. Después, se actualizan

Archivo	Contenido
`config.txt`	Una línea con la subruta fija donde se encuentran las fotos. Por ejemplo, `/users/X`.
`usuarios.txt`	La primera línea contiene la cantidad N de usuarios. Cada línea posterior contiene `nombre|contrasenia|correo|ciudad|pais|rutaFotoPerfil|fechaNacimiento|fechaInscripcion`. Los usuarios deben estar ordenados por nombre y no debe haber nombres repetidos, ya que se utiliza búsqueda binaria.
`amistades.txt`	La primera línea contiene la cantidad de registros. Cada registro posterior tiene el formato `usuarioA|usuarioB`. Cada amistad se almacena una sola vez.
`solicitudes.txt`	La primera línea contiene la cantidad de solicitudes. Cada registro posterior tiene el formato `emisor|receptor`.
`albums.txt`	Cada registro tiene el formato `usuario|album|privacidad`. La privacidad se representa mediante 0 para público, 1 para amigos y 2 para privado.
`fotos.txt`	Cada registro tiene el formato `usuario|album|nombreFoto|fecha|tamano|likes`. La ruta completa se construye a partir de la subruta configurada, el usuario, la carpeta `albums`, el álbum y el nombre de la foto.
`mensajes.txt`	Cada registro tiene el formato `emisor|receptor|fecha|leido|texto`. El campo `leido` toma los valores 0 o 1. El texto se ubica al final para permitir que contenga el carácter `|`.

Tabla 4: Formato de los archivos utilizados por el programa.

las dos posiciones correspondientes de la matriz y cada usuario se agrega a la lista de amigos del otro.

3. Las solicitudes cargadas se someten a las mismas validaciones que las solicitudes nuevas, con el fin de descartar registros repetidos o relaciones que no cumplan las reglas.

4. `CargadorDatos` procesa los álbumes, las fotos y los mensajes. Cada mensaje se crea una sola vez y se registra en la lista de enviados del emisor y en la lista de recibidos del receptor. Finalmente, las listas de mensajes se ordenan por fecha mediante el algoritmo de inserción.

5. Para iniciar sesión, se busca el nombre de usuario mediante búsqueda binaria y se compara la contraseña. Si se producen tres intentos fallidos, se regresa al menú inicial.

5.2. Selección aleatoria de fotos de perfil con prioridad

A cada amigo se le asigna un peso según la cantidad de amigos en común con el usuario actual: 2 si tiene hasta 10, 5 si tiene entre 11 y 20, y 7 si tiene más de 20. Estos pesos representan las proporciones relativas $1 : 2{,}5 : 3{,}5$ sin necesidad de utilizar valores decimales.

1. Se calcula la cantidad de amigos en común de cada amigo mediante la operación AND entre las filas correspondientes de la matriz de amistad.

2. Se suman los pesos de los candidatos y se genera un número aleatorio entre cero y la suma menos uno. Se recorren los candidatos acumulando sus pesos hasta que el acumulado supera el número generado; el candidato correspondiente queda seleccionado.

3. El peso del candidato seleccionado se establece en cero para impedir que vuelva a ser elegido. El procedimiento se repite hasta seleccionar tres amigos o agotar los candidatos.

4. Para cambiar el grupo, operación permitida una sola vez, se excluyen inicialmente los tres amigos anteriores. Si no existen suficientes candidatos diferentes, se completa el grupo con los anteriores, evitando repetir usuarios dentro del nuevo grupo.

5.3. Solicitudes de amistad

Antes de crear una solicitud, se comprueba que el destinatario no sea el propio usuario, que no exista una amistad previa, que el destinatario no haya enviado ya una solicitud y que el usuario actual no haya enviado otra al mismo destinatario.

Al aceptar una solicitud, se registra la amistad en ambas direcciones de la matriz, se incorpora cada usuario a la lista de amigos del otro y se elimina la solicitud. Al rechazarla, únicamente se elimina la solicitud recibida.

5.4. Sugerencias de amistad

1. Se crea un arreglo de N contadores inicializados en cero.
2. Para cada amigo del usuario actual, se recorren los amigos de ese amigo. Cuando un candidato no es el propio usuario ni uno de sus amigos, se incrementa su contador. Al finalizar, cada contador representa la cantidad de amigos en común del candidato con el usuario actual.
3. Se mantiene un arreglo ordenado de tamaño K. Cada candidato se inserta en la posición que le corresponde según su cantidad de amigos en común. Como $K \leq 20$, este procedimiento evita ordenar por completo los N candidatos.

5.5. Carrusel de fotos

El carrusel utiliza un índice i para recorrer las n fotos de un álbum. La siguiente foto se obtiene mediante $(i + 1)$ mód n, mientras que la anterior se obtiene mediante $(i - 1 + n)$ mód n. Estas operaciones permiten recorrer las fotos de forma circular.

El sistema almacena en cada foto la información del último usuario que dio "me gusta". Esta estrategia permite identificar la interacción más reciente, pero no representa por sí sola el historial completo de usuarios que han dado "me gusta". Por tanto, su alcance depende de las reglas de interacción implementadas en el programa.

5.6. Mensajes

- **Envío:** se verifica que el destinatario sea amigo y que el texto tenga entre 1 y 512 caracteres. Se crea un único objeto de mensaje y se incorpora, ordenado por fecha, a las listas de enviados y recibidos. Si el mensaje es el más reciente, puede insertarse al final sin desplazar los elementos anteriores.
- **Lectura de mensajes nuevos:** primero se muestran todos los mensajes no leídos junto con la información de sus emisores. En un segundo recorrido se actualiza su estado a leído.

- **Búsqueda por rango de fechas:** las fechas se solicitan en formato `AAAAMMDD`. La fecha inicial se interpreta a las 00:00 y la final a las 23:59, de modo que se incluyan ambos días completos. Se utiliza búsqueda binaria para localizar el primer mensaje cuya fecha sea mayor o igual al inicio y, desde esa posición, se recorren los mensajes hasta superar el límite final.

5.7. Política de guardado

La política de guardado busca reducir las escrituras innecesarias y limitar la pérdida de cambios ante una interrupción del programa.

Las operaciones se clasifican según su prioridad de persistencia. Los cambios que deben guardarse inmediatamente se escriben en los archivos correspondientes tan pronto como se producen. Los demás cambios pueden acumularse hasta alcanzar diez modificaciones, hasta que transcurran 120 segundos desde la primera modificación pendiente o hasta el cierre de sesión.

Solo se reescriben los archivos marcados como modificados. Antes de escribir, el contenido de cada archivo se construye completamente en memoria. Este procedimiento permite preparar la información antes de reemplazar el contenido del archivo, aunque no garantiza por sí solo la recuperación ante cualquier fallo durante la escritura.

6. Medición del consumo de recursos

Cada funcionalidad utiliza un nuevo objeto de `medidorrecursos`, que comienza con sus contadores inicializados en cero. Al finalizar, se presentan tres métricas: iteraciones contabilizadas, memoria estimada y uso de componentes externos.

- **Iteraciones:** se contabilizan las vueltas de los ciclos mediante variables locales y se acumulan en el medidor. También se consideran las operaciones indirectas incluidas en la instrumentación, como las búsquedas binarias, el cálculo de amigos en común, el conteo de bits, el ordenamiento de mensajes y los guardados automáticos desencadenados por la funcionalidad.
- **Memoria:** se estima el espacio ocupado por las estructuras del programa, incluidos la matriz, los usuarios, sus textos y listas, los álbumes, las fotos, las solicitudes, los mensajes, las variables locales y los parámetros por valor de la funcionalidad. En las listas dinámicas se considera la capacidad reservada. Se evita contar dos veces los textos almacenados dentro del propio objeto cuando no requieren asignación dinámica adicional. También se contemplan los arreglos temporales y los textos construidos para guardar los archivos, de acuerdo con el alcance de la instrumentación.
- **Componentes externos:** se registra el número de usos de funciones o servicios externos considerados por el medidor, como `rand` para las selecciones aleatorias, `time`/`localtime` para obtener la fecha y hora de los mensajes, y `chrono` para controlar los intervalos de guardado.

Las iteraciones constituyen una medida de las operaciones contabilizadas por la instrumentación implementada; no equivalen directamente al tiempo de ejecución. Del mismo modo, la memoria reportada es una estimación cuyo alcance depende de los elementos incluidos en el cálculo.

7. Resultados

El escenario de prueba contiene 1000 usuarios, 15839 amistades, 339 solicitudes, 2505 álbumes, 16310 fotos y 10099 mensajes.

Funcionalidad	**Iteraciones**	**Memoria (KB)**	**Componentes externos**
I. Carga de datos	**766.318**	**4.263**	**Ninguno**
II. Inicio de sesión y fotos de perfil	**1.512**	**4.263**	**rand × 3**
III. Envío y gestión de solicitudes	**34.026**	**5.114**	**chrono**
IV. Perfil de un amigo y carrusel	**50**	**4.263**	**Ninguno**
V. Sugerencias con $K = 12$	**4.277**	**4.267**	**Ninguno**
VI. Envío de mensajes	**10**	**4.263**	**time/localtime × 2; chrono × 2**
VI. Lectura de mensajes nuevos	**28**	**4.263**	**chrono**
VI. Búsqueda por rango de fechas	**14**	**4.263**	**Ninguno**

Tabla 5: Métricas registradas por el programa en el escenario de prueba.

La carga de datos es la operación con mayor número de iteraciones contabilizadas, con 766.318. Durante este proceso se leen y relacionan siete archivos que contienen, en conjunto, más de 46.000 líneas. Para cada amistad se realizan dos búsquedas binarias, se actualizan los bits correspondientes de la matriz y se incorpora cada usuario a la lista de amigos del otro. Adicionalmente, se ordenan las listas de mensajes de los usuarios.

Las demás funcionalidades operan sobre la red ya cargada en memoria y, de acuerdo con las mediciones obtenidas, presentan un número de iteraciones considerablemente menor.

La memoria reportada permanece prácticamente constante, alrededor de 4.263 KB, porque las mediciones se realizan con la mayor parte de los datos de la red ya almacenados. Las variaciones corresponden a los recursos temporales asociados a ciertas operaciones. En la funcionalidad V, la memoria aumenta a 4.267 KB por el arreglo temporal de 1000 contadores de sugerencias. En la funcionalidad III, aumenta a 5.114 KB debido al trabajo de guardado que construye en memoria el contenido de los archivos modificados.

La funcionalidad III registra 34.026 iteraciones, cifra que incluye el trabajo asociado al procesamiento de solicitudes y, según la instrumentación descrita, las operaciones de guardado que se ejecutan durante la prueba. La funcionalidad V registra 4.277 iteraciones porque recorre las amistades de los amigos del usuario para calcular las sugerencias.

Por su parte, las operaciones de mensajes y el carrusel presentan menos iteraciones en este escenario. La búsqueda por rango se beneficia de la búsqueda binaria para localizar el comienzo del intervalo, mientras que el recorrido posterior depende de la cantidad de mensajes comprendidos en el rango consultado.

Estos resultados corresponden al escenario de prueba utilizado y a las operaciones contabilizadas por el programa. No deben interpretarse como tiempos de ejecución ni como una comparación universal de rendimiento entre algoritmos.

8. Problemas de desarrollo

1. **Inclusiones circulares.** Las clases `Usuario` y `Mensaje` necesitan referencias mutuas. El problema se resolvió mediante declaraciones adelantadas, como `class Mensaje;`, `class Album;` y `class SolicitudAmistad;`, en `usuario.h`, mientras que las inclusiones completas se realizaron en los archivos de implementación cuando fueron necesarias.

2. **Saltos de línea en Windows.** Algunos archivos creados en Windows conservaban el carácter `\r` al final de ciertos campos. Para evitar errores de comparación y procesamiento, se añadió una función que elimina ese carácter durante la lectura.

3. **Duplicación de mensajes.** Al mantener copias independientes de un mensaje en las listas del emisor y del receptor, el estado de lectura podía quedar desincronizado. La solución fue crear un solo objeto y almacenar su puntero en ambas listas. Esta decisión requiere definir claramente la propiedad del objeto: únicamente el emisor es responsable de liberarlo, mientras que el receptor conserva una referencia no propietaria.

4. **Control de "me gusta".** Se incorporó en cada foto un campo que identifica al último usuario que dio "me gusta", sin añadir una lista de usuarios ni modificar el formato de los archivos. Esta solución reduce la información almacenada, pero no permite mantener un registro completo de las interacciones de todos los usuarios.

5. **Doble contabilización de memoria.** La medición inicial sumaba dos veces ciertos textos cortos al considerar su capacidad y el tamaño del objeto que ya los contenía. Se ajustó el cálculo para distinguir entre el almacenamiento interno de los objetos y las asignaciones dinámicas adicionales.

6. **Operaciones no contabilizadas.** Inicialmente, las iteraciones asociadas a la carga, el inicio de sesión y los guardados automáticos no se atribuían correctamente a las funcionalidades que las desencadenaban. Se incorporó un acumulador en `UdeaBook` para registrar ese trabajo indirecto.

9. Evolución de la solución

- **Diseño inicial (entrega del 9 de octubre):** se plantearon diez clases, una matriz de adyacencia, una lista dinámica genérica y la gestión de los objetos a cargo de las clases correspondientes. En esa etapa, los datos se guardaban únicamente al cerrar sesión.

- **Implementación:** se separaron las operaciones de lectura y escritura en `CargadorDatos` y `GuardadorDatos`. También se reemplazó la estrategia de guardado exclusivo al cerrar sesión por una política que distingue entre cambios con guardado inmediato y cambios acumulables.

- **Mensajes y fotos:** se pasó de mantener copias independientes de los mensajes a compartir un único objeto entre las listas de enviados y recibidos. Además, se incorporó un mecanismo para registrar información sobre el último usuario que dio "me gusta".

- **Medición de recursos:** se amplió la instrumentación para incluir el trabajo asociado a la carga de datos, el inicio de sesión, los guardados y el uso de componentes externos.

10. Conclusiones

- La matriz de adyacencia almacenada mediante bits permite representar las amistades de 1000 usuarios utilizando aproximadamente 125 KiB para el arreglo de bits. Además, permite consultar una relación de amistad en $O(1)$ y calcular amigos en común mediante operaciones entre palabras de 64 bits.
- Mantener los usuarios ordenados por nombre permite realizar búsquedas binarias con complejidad $O(\log N)$. Esta estrategia reduce la cantidad de comparaciones necesarias para localizar usuarios frente a una búsqueda secuencial, especialmente cuando aumenta el tamaño de la red.
- Compartir un único objeto `Mensaje` entre las listas de enviados y recibidos evita duplicar su contenido y permite mantener un estado de lectura común. Sin embargo, esta estrategia exige establecer reglas claras de propiedad y liberación de memoria para prevenir errores relacionados con el uso de punteros.
- La medición de las operaciones directas e indirectas permitió identificar la carga de datos como la funcionalidad con mayor número de iteraciones en el escenario evaluado. También evidenció que la persistencia de los cambios puede representar una parte importante del trabajo realizado por una operación, por lo que debe incluirse al analizar el rendimiento del sistema.
- La separación de responsabilidades entre las clases facilita organizar la solución y mantener sus componentes de manera independiente. No obstante, el rendimiento y la confiabilidad no dependen únicamente de elegir estructuras eficientes: también requieren controlar la propiedad de la memoria dinámica, validar los datos de entrada y garantizar que las operaciones de actualización mantengan la coherencia de la información.